

SEMANA 3 — DÍA 1

Estadística descriptiva y fundamentos matemáticos para análisis

Objetivo del día

Aplicar medidas básicas de estadística descriptiva a los datos limpios del Script 1 y reforzar fundamentos matemáticos útiles para análisis.

Teoría

1. Fundamentos matemáticos útiles para análisis

Antes de hacer estadística, es importante recordar que Python no solo maneja números “normales”, sino que tiene herramientas matemáticas que ayudan a evitar errores y a representar datos de forma más precisa. Estos conceptos no son complicados, pero sí muy útiles cuando analizamos datos reales.

1.1 Fracciones en Python (fractions.Fraction)

Cuando trabajamos con números decimales, Python usa **punto flotante**, lo cual puede generar errores como:

```
0.1 + 0.2 # Resultado: 0.30000000000000004
```

Esto no es un error de Python: es cómo funcionan las computadoras.

Para evitarlo, Python tiene el módulo:

```
from fractions import Fraction
```

Ejemplo:

```
from fractions import Fraction

x = Fraction(1, 3)
y = Fraction(2, 3)

print(x + y) # 1
```

¿Cuándo usar fracciones?

- Cuando necesitas **exactitud total** (por ejemplo, proporciones).
- Cuando un cálculo decimal te da resultados “raros”.
- Cuando quieres evitar errores acumulados.

¿Se instala?

No. Viene incluido en Python.

1.2 Notación científica

La notación científica permite escribir números muy grandes o muy pequeños de forma compacta:

```
G = 6.673e-11 # Constante gravitacional
```

6.673e-11 significa:

$[6.673 \times 10^{-11}]$

¿Cuándo usarla?

- Cuando tu API maneja valores muy grandes (p. ej., distancias, masas, energía).
- Cuando tus datos tienen muchos ceros y se vuelven difíciles de leer.
- Cuando quieres mostrar valores de forma más limpia.

¿Se instala?

No. Es parte del lenguaje.

1.3 Números complejos

Python maneja números complejos usando la letra j:

```
z = 2 + 3j
```

¿Para qué sirven?

- Señales
- Audio
- Imágenes
- Transformadas (Fourier)
- Simulaciones

¿Los usaremos en el PIA?

Probablemente no, pero es importante saber que Python los soporta de forma nativa.

1.4 Sistemas numéricos (binario, octal, hexadecimal)

Python puede convertir números a otras bases:

```
bin(10)    # '0b1010'
oct(10)    # '0o12'
hex(10)    # '0xa'
```

¿Cuándo sirve esto?

- Cuando trabajas con datos de sensores.
- Cuando tu API usa códigos hexadecimales (colores, identificadores).
- Cuando necesitas interpretar datos de bajo nivel.

¿Se instala?

No. Es parte del lenguaje.

2. Estadística descriptiva (librería statistics)

La estadística descriptiva sirve para **resumir** datos.

No predice, no modela, no hace magia: solo describe lo que tienes.

Python tiene una librería llamada statistics que facilita estos cálculos.

¿Se instala?

No. Viene incluida en Python.

```
import statistics as stats
```

2.1 Media (mean)

Promedio de los valores.

```
stats.mean([1, 2, 3]) # 2
```

2.2 Mediana (median)

Valor central cuando los datos están ordenados.

```
stats.median([1, 100, 2]) # 2
```

2.3 Moda (mode)

Valor que más se repite.

```
stats.mode([1, 2, 2, 3]) # 2
```

2.4 Desviación estándar (stdev)

Qué tanto se alejan los datos de la media.

```
stats.stdev([10, 12, 12, 15, 20])
```

2.5 Varianza (variance)

Dispersión al cuadrado.

2.6 Rango

Diferencia entre el valor máximo y mínimo.

```
max(lista) - min(lista)
```

3. ¿Cuándo usar cada medida?

3.1 Media → datos simétricos

Úsala cuando:

- No hay valores extremos
- Los datos están “parejos”

Ejemplo: alturas de personas.

3.2 Mediana → datos con valores atípicos

Úsala cuando:

- Hay valores muy grandes o muy pequeños
- La media se distorsiona

Ejemplo: ingresos económicos.

3.3 Moda → categorías

Úsala cuando:

- Quieres saber qué categoría es más común
- Los datos no son numéricos

Ejemplo: tipo de Pokémon, clima, color, categoría.

3.4 Desviación estándar → dispersión

Úsala cuando:

- Quieres saber qué tan “separados” están los datos
- Quieres medir variabilidad

Ejemplo: variación de temperatura, precios, velocidades.

3.5 Rango → amplitud

Úsala cuando:

- Quieres una medida rápida de variación
- No necesitas precisión fina

Ejemplo: diferencia entre el valor más alto y más bajo.

4. *Cómo leer archivos en Python (JSON y TXT)*

Leer archivos es una de las habilidades más importantes en análisis de datos.

Cuando guardas un archivo, la información queda escrita en tu computadora.

Pero para usarla en tu programa, necesitas **abrirlo**, **leerlo** y **convertirlo** a un formato que Python pueda manipular.

Python puede leer muchos tipos de archivos:

- .json → datos estructurados
- .txt → texto plano
- .csv → tablas
- .xml → datos jerárquicos
- .png, .jpg → imágenes

En el PIA trabajaremos principalmente con **JSON** y **TXT**.

5. *Leer archivos JSON*

JSON significa **JavaScript Object Notation**.

Es un formato muy común para APIs porque:

- Es fácil de leer
- Es fácil de escribir
- Representa datos como diccionarios y listas
- Python lo convierte automáticamente a estructuras nativas

Ejemplo de JSON:

```
{
  "nombre": "Pikachu",
  "altura": 4,
  "peso": 60
}
```

5.1 ¿Qué librería se usa?

Python incluye la librería json **por defecto**.

No se instala con pip.

```
import json
```

5.2 Cómo leer un archivo JSON

Supongamos que tienes:

```
data/clean/data_clean.json
```

Código:

```
import json

with open("data/clean/data_clean.json", "r") as f:
    data = json.load(f)
```

¿Qué hace este código?

- `open()` → abre el archivo
- `"r"` → modo lectura (read)
- `json.load()` → convierte el contenido del archivo a un **diccionario de Python**

¿Qué obtienes?

Si el JSON contiene un objeto → obtienes un **diccionario**

Si contiene una lista → obtienes una **lista de diccionarios**

Ejemplo:

```
print(type(data))
print(data)
```

5.3 Leer una lista de objetos JSON

Si tu archivo tiene varios registros:

```
[
    {"nombre": "Pikachu", "altura": 4},
    {"nombre": "Bulbasaur", "altura": 7}
]
```

Python lo leerá como:

```
[
    {"nombre": "Pikachu", "altura": 4},
    {"nombre": "Bulbasaur", "altura": 7}
]
```

Puedes recorrerlo:

```
for item in data:
    print(item["nombre"], item["altura"])
```

6. Leer archivos TXT

Los archivos .txt contienen texto plano.

Son útiles para:

- Guardar logs
- Guardar mensajes
- Guardar listas simples
- Guardar resultados no estructurados

6.1 Leer un archivo TXT línea por línea

```
with open("data/raw/response.txt", "r") as f:
    for linea in f:
        print(linea)
```

6.2 Leer todo el contenido de un TXT

```
with open("data/raw/response.txt", "r") as f:
    contenido = f.read()

print(contenido)
```

6.3 Leer un TXT como lista de líneas

```
with open("data/raw/response.txt", "r") as f:
    lineas = f.readlines()

print(lineas)
```

7. Diferencias entre leer JSON y TXT

Tipo de archivo	Contenido	Resultado en Python
JSON	Datos estructurados	Diccionario o lista
TXT	Texto plano	Cadena o lista de líneas

JSON es ideal para datos de APIs.

TXT es ideal para texto libre.

8. Errores comunes al leer archivos

✗ Ruta incorrecta

Solución: verificar carpetas y nombres.

✗ Archivo no existe

Solución: crear carpetas con `os.makedirs()`.

✗ JSON mal formateado

Solución: validar antes de guardar.

✗ Usar `json.loads()` en lugar de `json.load()`

- `json.load()` → lee desde archivo
- `json.loads()` → lee desde cadena



Ejemplos guiados

Ejemplo 1: Cargar datos limpios

```
import json

with open("data/clean/data_clean.json") as f:
    data = json.load(f)
```

Ejemplo 2: Extraer una variable numérica

```
alturas = [item["altura"] for item in data]
```

Ejemplo 3: Calcular estadísticas

```
import statistics as stats

media = stats.mean(alturas)
mediana = stats.median(alturas)
rango = max(alturas) - min(alturas)

print("Media:", media)
print("Mediana:", mediana)
print("Rango:", rango)
```

Ejemplo 4: Leer datos limpios para análisis

```
import json
```

```
with open("data/clean/data_clean.json") as f:
    data = json.load(f)

alturas = [item["altura"] for item in data]
```

Ejemplo 5: Leer datos crudos para depuración

```
with open("data/raw/response.txt") as f:
    print(f.read())
```

✖ Resumen

- **Media** → promedio
- **Mediana** → centro real
- **Moda** → lo más común
- **Rango** → amplitud
- **Desviación estándar** → qué tanto varían
- **Fracciones** → exactitud
- **Notación científica** → números extremos
- **Complejos** → no los usaremos, pero existen
- **Sistemas numéricos** → útil para códigos o sensores
- Para leer JSON → usa `json.load()`
- Para leer TXT → usa `read()`, `readlines()` o un ciclo
- JSON se convierte en diccionarios y listas
- TXT se convierte en texto

Actividad

Cada equipo debe:

1. Cargar (leer) su archivo limpio (`data/clean/data_clean.json`).
2. Identificar **al menos 2 variables numéricas** de su API.
3. Calcular **media, mediana y rango**.
4. Crear el archivo `analysis.py` con funciones:
 - `calcular_media(lista)`
 - `calcular_mediana(lista)`
 - `calcular_rango(lista)`
5. Imprimir resultados en consola.

🏠 Tarea

- Terminar la lectura y análisis estadístico en un nuevo script que será el “script 2” de su proyecto.
- Subirlo a GitHub.
- Elegir qué variables graficarán mañana.